

一种自我改进的编码智能体

Maxime Robeyns,
University of Bristol
maxime@igent.ai

Martin Szummer
iGent AI
szummer@igent.ai

Laurence aitchison
University of Bristol
laurence.aitchison@bristol.ac.uk

摘要

大语言模型（Large Language Models, LLMs）的最新进展激发了人们对部署大语言模型智能体以承担现实世界任务的兴趣。大语言模型通常被部署在智能体系统中：即编排大语言模型调用并为其提供工具的代码。我们证明，一个配备了基本编码工具的智能体系统能够自主地编辑自身，从而提升其在基准任务上的性能。我们发现，在 SWE Bench Verified 的一个随机子集上，性能提升幅度从 17% 到 53% 不等，同时在 LiveCodeBench 以及合成生成的智能体基准上也取得了额外的性能提升。我们的工作代表了在智能体系统的自动化与开放式设计方面的一项进展，并展示了一种由大语言模型反思和代码更新驱动的、数据高效且非基于梯度的学习机制。

1 引言

大语言模型最近在一系列领域和任务上取得了令人瞩目的进展 [2, 14, 29]。然而，为了将这些大语言模型应用于现实世界的应用中，必须将大语言模型封装在代码中，以编排它们并暴露允许模型采取行动的工具。这些能够采取行动的大语言模型被称为智能体，而更广泛的系统则称为智能体系统。

这些智能体系统通常通过结合提示策略和组合不同大语言模型输出的方法，在基准性能上比“普通”大语言模型表现出显著的提升 [43, 46, 5]。早期的例子包括最佳 N 采样和简单的提示策略，如思维链 [20]。然而，更复杂的方案在从模型中获取期望行为和性能提升方面已显示出成功，例如 STaR[45]、思维树 [42]、思维图 [3]、大语言模型辩论 [10]、迭代自我精炼 [23]、专家提示 [22] 等众多方法。Schulhoff 等人 [34] 的全面综述展示了迄今为止大量人工创建的策略。

近期编码智能体的改进引发了一个问题：这些智能体能否自主地修改和改进自身代码，例如发现新的提示方案或工具，而无需人工设计和实现。本文认为，这种完全自引用的元智能体编程在今天可行的，并为手工编排器的临时试错方法提供了一种合理的替代方案，后者可能只探索了解空间的一小部分。智能体系统自动设计（Automated Design of Agentic Systems, ADAS）[16] 的最新工作使用元智能体来优化智能体实现。然而，Hu 等人 [16] 的工作并非自我改进，因为存在两个独立的智能体：执行任务的目标智能体和改进目标智能体的元智能体。自我改进系统的一个动机是，编码能力的提升可以在后续的改进步骤中得到利用，有望实现复合增长。

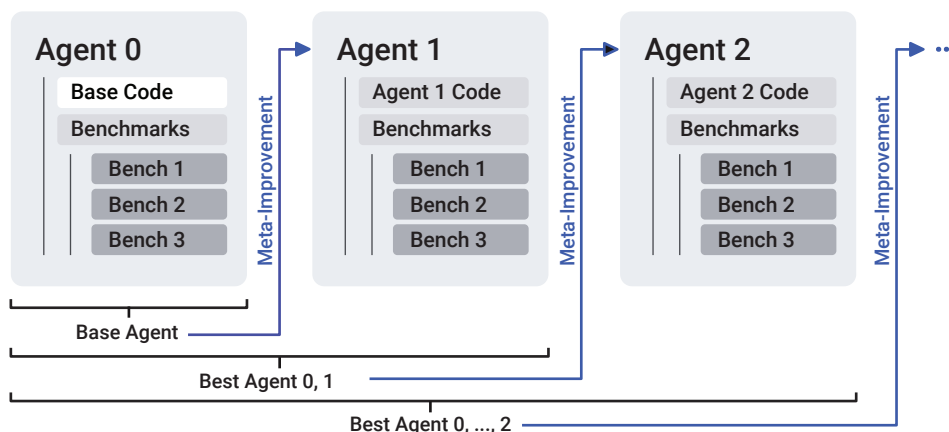


图 1: 元智能体循环: 智能体从支持初始自我改进所需的最少代码开始, 然后遵循基准测试和元改进的序列。

本文的贡献如下:

- 一种自我改进编码智能体 (Self-Improving Coding Agent, SICA), 消除了元智能体与目标智能体之间的区别, 能够编辑自身代码库, 以在成本、速度和基准性能方面改进自身。
- 实证证据表明, 自引用智能体能够有效改进其自身实现; 我们发现, 在 SWE-Bench Verified 的随机子集上, 性能从 17% 提升至 53%, 即使考虑了安全约束和资源效率。
- 我们向社区分享了自我改进编码智能体 (SICA) 的实现。SICA 使用标准 Python 实现, 无需领域特定语言, 为构建新的 SICA 系统以及那些希望在大语言模型上针对工具使用和其他智能体任务进行后训练的研究者提供了一个参考智能体框架。

我们的代码可在 https://github.com/MaximeRobeyns/self_improving_coding_ 智能体处获取。

2 相关工作

开发和优化智能体系统的传统方法是人工设计智能体架构和提示技术。典型例子包括用于改进推理的思维链 (Chain-of-Thought) 提示 [39]、自我精炼 [23] 和自我反思 [37]、工具使用框架 [33] 以及各种组合式智能体系统 [38, 1]。尽管这些手工设计的方法取得了显著成果, 但它们需要大量人力, 并且可能遗漏通过自动搜索才能发现的有用模式。

另一个方向侧重于使智能体能够学习可复用技能并持续自我改进。MaestroMotif[19] 利用大语言模型反馈来学习技能奖励, 并通过代码生成组合技能。这建立在早期关于内在动机强化学习 [6] 和自驱动智能体 [8] 的工作基础上, 这些工作开发了由内在动机驱动的技能库, 同时也借鉴了开放性研究 [48, 12] 中使用大语言模型识别有趣且有用的探索方向的工作。

多种方法利用大语言模型通过自然语言交互来优化智能体行为。OPRO[41] 和 Promptbreeder[13] 专注于通过语言优化提示。其他研究探索了使用大语言模型来批判和改进智能体行为 [18]、生成训练课程 [21] 或为强化学习提供自然语言反馈 [31]。由于我们的智能体可以编辑其整个代码库, 这包括调整自身提示以优化智能体任何部分行为的能力。

近期工作开始探索自动设计和优化智能体系统的方法。AgentSquare[35] 提出了一个模块化设计空间，将智能体组件抽象为规划、推理、工具使用和记忆模块，从而允许对模块组合进行自动搜索。AlphaEvolve[28] 旨在使用编码智能体进行科学发现并优化计算基础设施，其结果可能加速支撑 AlphaEvolve 自身的大语言模型的训练。这些工作采用的方法比我们的方法更侧重于结构化的进化搜索。

或许最密切相关的前期工作始于自适应智能体系统 (ADAS) [16]。ADAS 采用一个执行实际任务的目标智能体，以及一个改进目标智能体的元智能体。因此，ADAS 并非自我改进（因为元智能体改进的是目标智能体，而非自身）。此外，在 ADAS 中，元智能体仅编辑一个单一的前向函数，该函数使用一种精心设计的领域特定语言编写，旨在使表达不同的提示方案变得非常直接。相比之下，本文提出的自我改进编码智能体是完全自我改进的（即元智能体与目标智能体之间没有区别），并且它操作于智能体的完整 Python 代码库之上。

当然，我们预期首批真正自我改进的智能体将是编码智能体，因为智能体是用代码编写的。自然的方法是先从一个基础的编码智能体开始，该智能体能够打开、关闭、编辑文件，在终端中运行命令等，然后将其置于自我改进循环中启动。我们认为本文提出的自我改进编码智能体是首个此类工作。然而，有两篇论文声称实现了自我改进智能体，但它们并未在编码环境中进行评估，因为它们没有考虑“完整”的编码智能体。首先，哥德尔智能体 (Gödel Agent) [44] 具有特定工具（如 `action_adjust_logic` 和 `action_read_logic`），允许在智能体运行时修改其小部分。因此，它不是传统意义上的通用编码智能体。因此，与 ADAS 一样，它是在语言理解和数学基准 (DROP [11]、MGSM [36]、MMLU [15] 和 GPQA [32]) 上进行评估的，而非编码基准。其次，泽利克曼等人 (Zelikman et al.) [47] 提出了一种用于递归自我改进代码生成的自我教导优化器。同样，这也不是一个通用编码智能体，不具备处理任意软件工程任务的能力，因此，它仅在算法任务上进行自我改进和评估，例如带噪声的奇偶学习 [4]、字符串网格距离和 3-SAT 问题。

3 方法

SICA 的主运行循环与 Hu 等人提出的 ADAS 循环 [16] 相似，如图 1 所示。具体而言，SICA 和 ADAS 都维护一个包含先前智能体及其基准测试结果的档案。第一步，SICA 从档案中选取截至当前表现最佳的智能体作为元智能体，并指示其查阅档案、识别改进点并加以实现。高层方案如算法 1 所示。注意，这与 ADAS 不同，ADAS 的元智能体是固定的，因此不会从档案中选取元智能体（ADAS 中的档案是目标智能体的档案，而非元智能体的档案）。第二步，ADAS 和 SICA 都会在一组评估基准上评估新智能体，并存储结果。

我们通过效用函数定义“最佳”表现智能体，该函数包含了对优秀智能体的常见期望：归一化基准性能得分 $p_{\text{score}} \in [0, 1]$ 、以秒为单位的墙钟时间 p_{time} 以及美元成本 p_{cost} 。基础效用计算如下

$$U = w_{\text{score}}p_{\text{score}} + w_{\text{cost}}(1 - \min(1, p_{\text{cost}}/\$10)) + w_{\text{time}}(1 - \min(1, p_{\text{time}}/300 \text{ s})) \quad (1)$$

其中我们将系数设为 $w_{\text{score}} = 0.5$, $w_{\text{cost}} = 0.25$ 和 $w_{\text{time}} = 0.25$ 。我们将最差成本限制为每个问题 10 美元，并设置 300 秒的超时，超时后取消智能体。为了对超时前完成的工作给予部分认可，我们按如下方式计算最终效用，超时惩罚为 $\tau = 0.5$ ：

$$U_{\text{final}} = \begin{cases} U(1 - \tau), & \text{if timed out} \\ U, & \text{otherwise} \end{cases} \quad (2)$$

注意，由于我们不进行任何权重更新，该数值分数仅用于选择下一个元智能体以及下一次迭代的基础智能体。

我们首先描述初始编码智能体。接着描述基准运行器框架，以及该框架如何自然地允许我们创建自指任务（即改进编码智能体的任务）。

3.1 初始智能体

我们关注的是具有实际意义且能够自我改进的编码代理。为此，我们从一个复杂的编码代理入手，该代理配备了多种工具、子代理和监督机制。当然，这些工具、子代理和监督机制本身都可以进行自我改进；但在此我们仅描述其初始状态。

我们设计了一套工具，用于打开（即添加到上下文中）和关闭文件、覆盖文件、执行 shell 命令以及一个简单的计算器工具。我们还包含了基本的工具，用于向基准问题提交最终答案、将子代理调用的结果返回给其调用者，以及提前退出子代理调用。最后，我们提供了一个“归档分析”工具，该工具有助于提取归档中每个代理迭代的高级摘要统计信息。我们在附录 C 中更详细地描述了工具调用机制。

接下来，我们包含了几个子代理。这些子代理只是由现有代理调用的代理，类似于工具调用，通过接受输入参数并返回字符串值。引入这种抽象有两个目的。第一个目的是确保基准任务能够完全自主地运行，无需人工指导和引导。我们向主入口点代理提供问题陈述（其完整提示在附录 A 中给出），然后该代理分解任务，委派并向子代理提问。第二个目的是管理扩展代理运行的上下文长度。元改进任务具有挑战性，主代理可能需要大约一个小时才能完成一次元改进迭代。子代理调用允许在代理返回时自然地丢弃用于子任务的上下文，子代理返回的值在主代理的上下文中提供了该子任务的摘要。最初，我们包含了三个代理：编码代理、问题求解代理和推理代理。这些代理的完整提示在附录 A.1 中给出。

最后，我们在初始代理中提供的一个重要组件是异步监督者，它能够实现扩展的自主操作，同时保持任务专注，避免循环或其他阻碍进展的行为。这是一个大语言模型（LLM），在并发线程中定期运行，被提示判断是否存在任何病态行为或偏离原始任务的情况，并可以通过将消息内联到助手上下文中来通知当前运行的代理（或其任何父代理）进行干预，或者在严重情况下，如果代理偏离太远或似乎陷入困境，则取消代理的执行。监督者默认每 30 秒调用一次，或在关键事件时调用。

该监督器被提供系统状态的全面文本表示（截断示例见附录 B），其中冗长内容被截断以限制上下文。这包括代理与子代理执行的“调用图”，以及大语言模型消息、工具调用与响应、子代理调用与响应的事件流，这些均与每个代理调用相关联。这种对系统行为的广泛表示，使监督器能够识别模式，并检测先前代理调用所完成的重复工作。完整的监督器提示见附录 A.2。

代理上下文的结构（例如包含打开文件内容，而不仅仅是提示）至关重要。在初始编码代理中，上下文结构如图 3 所示。首先呈现包含代理定义（来自附录 A）的系统提示，列出代理可用工具的定义以及可调用的子代理。系统提示以系统信息结尾，例如如何跳出代理循环并返回调用进程的指令。

LLM Context Window



图 2：大语言模型上下文窗口结构。

Algorithm 1: Self-Referential Agent Improvement.

Input: Evaluation benchmarks $B = \{b_j : j = 1, \dots, m\}$, iteration count $n \in \mathbb{N}$

Output: Improved agent system A_n

- 1 Initialise agent A_0
 - 2 **for** $i = 0, 1, \dots, n - 1$ **do**
 - 3 Evaluate A_i on benchmarks B and store in p_i
 - 4 Run A_i to generate A_{i+1} , using the archive of $A_{\leq i}$ and $p_{\leq i}$, where $\hat{i} = \arg \max_{j \in \{0, \dots, i\}} p_j$
 - 5 **return** A_n
-

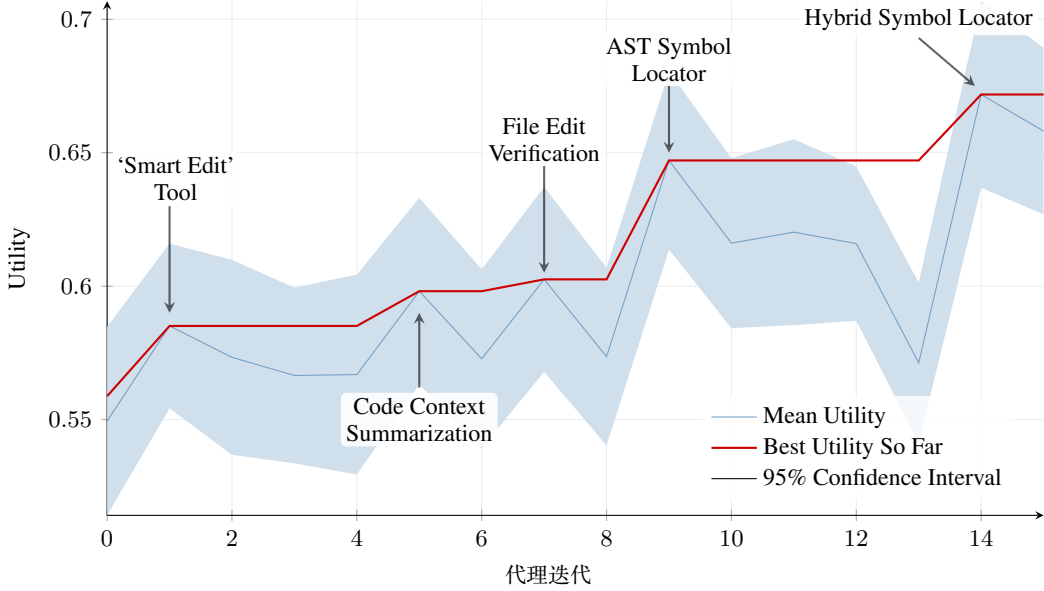


图 3: 跨迭代的性能表现。关键改进处标注了相应的工具或代理修改。

接下来是“核心提示”，它被设置为聊天模板格式中的第一条用户消息¹，包含调用者指定的待解决问题陈述（调用者可能是调用代理的用户，或调用子代理的代理）。此处还插入了代理已打开文件的视图，以及当前工作目录的表示。

最后，大语言模型代理上下文的其余部分是运行中的助手消息。这包含关于问题的思维链推理时段，其间穿插工具调用及其响应、子代理调用及其响应，以及来自异步监督器系统的任何通知或评论。

大语言模型上下文以这种方式布局，以便追加新内容，从而保留键值缓存，有利于降低工具调用之间的延迟和成本。对于基础代理，我们采用一种简单方案，将文件编辑追加为差异，并在累积一定数量后定期重新整合到过时的文件视图中。代理可以编写更复杂的键值缓存保留方案以降低成本。

4 实验与结果

我们首先展示一个包含基准集中相对长期任务的运行。我们纳入了 SWE 基准验证集（SWE Bench Verified）[7]，该验证集测试问题分解、代码导航以及快速高效的能力。

我们使用聊天模板中的用户消息角色作为核心提示，并非出于设计考虑，而是因为我们使用的是模型提供商应用程序编程接口（API）的聊天端点。如果针对智能体应用专门微调大语言模型（LLM），可以选择一个更恰当命名的节分隔符，例如<core>。

表 1: 每次迭代的性能: 基准准确率数字分别对应一个合成文件编辑基准和一个符号定位 (代码库导航) 基准、SWE 基准验证集 (SWE-Bench Verified) 以及 LiveCodeBench。我们还展示了所有基准中所有问题实例的平均度量, 以比较智能体迭代之间相对的“每个问题”成本。

Iter	Benchmark Accuracy				Average Metrics (per benchmark problem)			
	File	Sym	SWE-Bv	LCB	Cost (\$)	Time (s)	Tokens (M)	% Cached
0	0.82	0.35	0.17	0.65	1.91	130.2	0.24	32.5
1	0.87	0.32	0.14	0.62	1.71	123.8	0.24	33.6
2	0.92	0.31	0.17	0.58	2.45	151.4	0.26	34.3
3	0.82	0.33	0.22	0.64	1.84	126.9	0.29	31.9
4	0.88	0.31	0.38	0.54	2.70	148.3	0.26	30.9
5	0.89	0.31	0.30	0.59	2.17	134.8	0.23	36.9
6	0.96	0.31	0.37	0.64	2.21	134.1	0.25	36.1
7	0.92	0.35	0.33	0.58	2.15	134.9	0.23	38.2
8	0.93	0.33	0.27	0.64	2.03	128.5	0.26	36.4
9	0.88	0.40	0.47	0.61	2.03	126.3	0.27	38.3
10	0.87	0.41	0.46	0.66	1.77	107.0	0.22	40.9
11	0.89	0.43	0.44	0.70	2.25	129.6	0.27	36.9
12	0.91	0.38	0.44	0.64	1.58	103.9	0.26	39.0
13	0.86	0.40	0.27	0.61	1.66	113.3	0.29	37.2
14	0.94	0.34	0.53	0.67	2.20	117.1	0.25	37.2
15	0.91	0.40	0.51	0.71	1.70	114.5	0.30	36.3

文件编辑, 我们从中选取一个固定的随机子集, 包含 50 个问题。我们还纳入了来自 LiveCodeBench [26] 的 50 个随机问题, 这些问题类似于竞赛编程, 通常包含更多的理论推理²。

我们还纳入了两个示例性的“合成”基准任务, 这些任务完全在智能体的代码库内定义和策划。第一个是文件编辑基准, 其构建方式为: 克隆 SWE 基准中使用的仓库, 在连续提交中寻找“有趣的” (即非平凡、也非完全覆盖的) 文件编辑, 将检出到第一次提交的仓库挂载到工作目录中, 并提示智能体编辑文件, 使其与第二次提交的目标内容相匹配。智能体的评分依据是最终文件内容与目标内容的接近程度, 同时效用函数中还考虑了时间、美元成本和令牌消耗。第二个是代码库导航问题, 其构建方式同样为克隆一些 Python 仓库, 从中识别“有趣的”符号, 以及项目中的对该符号的至少一处引用。基准任务要求智能体定位该符号的定义, 并以示例引用中的格式 `path/to/file.py:line_num:column_num` 返回位置。

我们在智能体系统中大多数智能体使用 Sonnet 3.5 (v2) [2] 运行, 但有一个“推理智能体”例外, 它使用 o3 mini [30]。虽然基础智能体系统可以用更小、更便宜的模型运行, 而且我们的许多基准任务可以由这类模型解决, 但元改进任务是一个复杂的长期任务, 需要强大的模型。15 次迭代运行的应用程序编程接口成本约为 7,000 美元。

结果如图 3 所示, 更详细的迭代级性能数据见表 1。

数据呈现出有前景的进展。对于文件编辑基准, 智能体最初只具备覆盖文件的能力, 速度慢且令牌效率低, 它很快实现了基于差异和范围的、自称为“智能”的编辑工具。对于符号导航任务, 智能体在第 9 次迭代时实现了一个基于抽象语法树的简单符号定位器 (可能利用了我们的基准主要以 Python 为基础这一事实), 这带来了良好的改进, 也反映在其他任务中。³ SWE Bench Verified 子集的性能有了明显提升

²SWEBench-Verified 和 LiveCodeBench 均采用 MIT 许可证³ 注意, 我们将符号定位器基准中的低分归因于数据质量差——例如, 一些目标符号指向未安装的外部库。

在整个运行过程中，智能体框架不断改进。LiveCodeBench 分数有细微改善，但不如其他一些基准那么显著。

4.1 推理任务上的性能

我们还评估了自指智能体系统在更侧重推理的领域中改进任务性能的有效性。我们用基准集中的两个问答任务进行了另一项实验：AIME 2024 和 GPQA Diamond [32]，结果如图 4 所示。

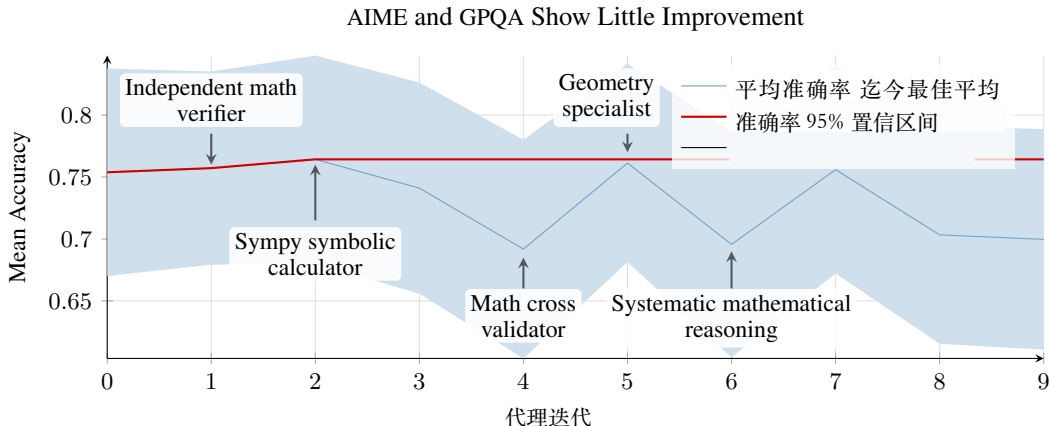


图 4：智能体框架饱和现象：当模型本身（如 o3-mini-high）已经表现良好时，智能体系统所能带来的收益微乎其微。

此处结果显示改进幅度较小，并凸显了基础模型与脚手架系统之间的重要相互作用。图 4 中使用的第 0 次迭代基础智能体系统采用了 Sonnet 3.5，并配备了一个使用 o3-mini 的“推理”子智能体。o3-mini 模型在“高”推理努力下，在 AIME 和 GPQA Diamond 上分别取得 87% 和 79% 的得分，而整个智能体系统在这两个基准上的平均得分为 76%。

通过检查执行轨迹，我们观察到在许多运行中，主智能体仅仅将问题委托给基于 o3-mini 的推理智能体，并未充分利用智能体系统的全部能力。对于诸如 o1 [29]、o3-mini 或 DeepSeek 的 R1 [9] 这类“推理模型”，我们怀疑引入旨在诱导推理行为的粗糙组件（如图 4 中第 4 次和第 6 次迭代所包含的组件）实际上可能会打断在智能体系统之外训练的推理模型的思维链，从而导致性能下降。我们期待未来工作能够将“智能体”大语言模型与智能体系统的设计联合训练，以期更好地发挥两者的优势。

5 结论

我们提出了 SICA，一个自我改进的编码智能体。我们的结果表明，通过智能体系统编辑自身代码库来实现自我改进确实是可行的，尤其是在更长周期的“智能体”任务中，基础大语言模型能够从脚手架系统的结构和指导中获得更多收益——例如在 SWE-Bench Verified 的随机子集上，性能从 17% 提升至 53%，同时平均每个问题的耗时也略有下降。我们提出的框架既为自动化智能体系统设计提供了一种实用的方法，也为未来联合训练基础模型与其智能体系统奠定了基础。

5.1 局限性

我们首次尝试构建自我改进的编码智能体并非没有局限性。一个关键难点在于，让基于大语言模型的智能体在每个元改进步骤中自主提出真正新颖、创新、可行且有趣的修改想法，这一主题在开放式学习文献中已有评论 [27, 40, 17]。采纳一个糟糕想法所带来的代价

导致“品味”不佳的原因在于，冗长的智能体编辑步骤之后，还需进行成本更高的基准测试运行。虽然失败的迭代仍保留在存档中，原则上充当反面示例，但我们发现，初始的功能构想往往会作为同一主题的变体，深刻影响后续的功能构想。这种路径依赖可能导致智能体运行结果的方差增大；低质量的初始功能建议（例如，执着于缓存已打开的文件）通常会降低后续功能建议的质量。

我们还注意到，在优化智能体运行时间和成本的过程中，我们相对较短的 5 分钟超时（以及较小程度上每个问题的成本限制）导致初始基准分数可能低于底层语言模型（例如 Sonnet 3.5 v2）的预期水平，尤其是对于像 SWE-Bench 这样较长周期的基准任务。实际上，我们所报告的初始改进很大程度上源于加速文件编辑和降低智能体成本的功能，而这些在传统上并未在 SWE-Bench 的数值中得到同等程度的体现。

5.2 未来工作

我们的方法体现了一种非基于权重的学习机制，通过优化模型所依赖的提示词，以及通过编辑底层代码来编排大语言模型调用方式，从而更新系统行为。从某种意义上说，这种方法样本效率非常高，对结果存档的详细检查能够指导每一步进行实质性修改。然而，我们也认识到将基础模型的权重与智能体系统联合更新的潜力。AlphaEvolve [28] 的工作证明了这一原理的可行性，而在每次迭代中更明确地微调基础模型，使其能够与新创建的工具和脚手架协同工作，可能会带来丰硕成果。

虽然我们在相对较短的运行中尚未达到基准饱和的程度，但我们预计未来的系统可能会从静态基准集中学无可学。解决这一问题的一种途径是自动设计和筛选基准，正如 [12] 的工作中初步展示的那样，其中强化学习环境通过代码进行设计。在我们的工作中，通过手动提示，智能体完全自主地构建了两个合成基准（文件编辑基准和符号导航基准），这表明 SICA 具有足够的灵活性，能够支持为自身创建新的基准和评估。

6 安全考量

对于设计为自我改进的系统所带来的安全影响，必须予以审慎考量。尽管我们的自我改进编码智能体在特定智能体任务中展现出性能提升，但此类纯粹基于脚手架的系统（如图 4 所示）似乎存在可达成目标的极限，因其并不修改模型权重。若系统同时更新模型权重与脚手架，则递归自我改进系统的风险预计将更高。我们着重指出两项关键机制以缓解潜在风险。

首先，此类系统中一项关键的安全缓解措施是可观测性：即人类能够且确实对潜在自我改进系统的所有方面进行监督。因此，我们将 SICA 架构设计为具备复杂而用户友好的可观测性机制，为每一步的思维链、动作及子智能体调用提供丰富的洞见。这些机制包括一个交互式网页接口，便于对智能体系统中发生的所有事件进行轻松监督，以及一个异步的基于大语言模型的监督者，该监督者可被提示将各种行为视为取消智能体的依据，并且可使用与运行中智能体不同的大语言模型。关于监督者判断的示例及其在每次异步调用中所获得的控制权，请参见附录 A.2。鉴于模型权重未更新，这使得模型在思维链中编码诸如速记消息 [24, 25] 等行为变得困难但并非不可能，我们认为这提供了强有力的安全缓解。然而，对于模型权重同样被更新的递归自我改进系统，仅靠可观测性机制显然不足，对安全机制的进一步研究显然必要。

第二个安全缓解的机遇内在于迭代式、基于评估的更新流程。尽管我们的初始基准集侧重于衡量我们希望提升的能力，但完全可以在该基准集中纳入与安全相关的评估，以便在每个智能体作为元智能体进入下一次迭代之前对其进行验证。

我们认为这项工作不会产生任何重大、直接的不利社会后果。我们的目标要么聚焦于改进代码编辑的机制，要么聚焦于通过更长周期的编码任务提升多步推理的有效性。

参考文献

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J. Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances, August 2022. URL <http://arxiv.org/abs/2204.01691>.
- [2] Anthropic. Introducing Claude 3.5 Sonnet. URL <https://www.anthropic.com/news/claude-3-5-sonnet>.
- [3] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michal Podstawski, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of Thoughts: Solving Elaborate Problems with Large Language Models, August 2023. URL <http://arxiv.org/abs/2308.09687>.
- [4] Avrim Blum, Adam Kalai, and Hal Wasserman. Noise-tolerant learning, the parity problem, and the statistical query model. *Journal of the ACM (JACM)*, 50(4):506–519, 2003.
- [5] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks, October 2023. URL <http://arxiv.org/abs/2211.12588>.
- [6] Nuttapon Chentanez, Andrew Barto, and Satinder Singh. Intrinsically Motivated Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 17. MIT Press, 2004. URL https://papers.nips.cc/paper_files/paper/2004/hash/4be5a36cbaca8ab9d2066debf4e65c1-Abstract.html.
- [7] Neil Chowdhury, James Aaung, and Chan Jung Shern. Introducing SWE-bench Verified, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- [8] Cédric Colas, Tristan Karch, Olivier Sigaud, and Pierre-Yves Oudeyer. Autotelic Agents with Intrinsically Motivated Goal-Conditioned Reinforcement Learning: A Short Survey. *Journal of Artificial Intelligence Research*, 74:1159–1199, July 2022. ISSN 1076-9757. doi: 10.1613/jair.1.13554. URL <https://www.jair.org/index.php/jair/article/view/13554>.
- [9] DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, January 2025. URL <http://arxiv.org/abs/2501.12948>.
- [10] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving Factuality and Reasoning in Language Models through Multiagent Debate, May 2023. URL <http://arxiv.org/abs/2305.14325>.
- [11] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *arXiv preprint arXiv:1903.00161*, 2019.
- [12] Maxence Faldor, Jenny Zhang, Antoine Cully, and Jeff Clune. OMNI-EPIC: Open-endedness via Models of human Notions of Interestingness with Environments Programmed in Code, October 2024. URL <http://arxiv.org/abs/2405.15568>.
- [13] Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. October 2023. URL <https://openreview.net/forum?id=HKkiX3Zw1>.
- [14] Google. Introducing Gemini 2.0: Our new AI model for the agentic era, December 2024. URL <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/>.
- [15] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

- [16] Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems, August 2024. URL <http://arxiv.org/abs/2408.08435>.
- [17] Edward Hughes, Michael Dennis, Jack Parker-Holder, Feryal Behbahani, Aditi Mavalankar, Yuge Shi, Tom Schaul, and Tim Rocktaschel. Open-Endedness is Essential for Artificial Superhuman Intelligence, June 2024. URL <http://arxiv.org/abs/2406.04268>.
- [18] Martin Klissarov, Pierluca D’Oro, Shagun Sodhani, Roberta Raileanu, Pierre-Luc Bacon, Pascal Vincent, Amy Zhang, and Mikael Henaff. Motif: Intrinsic Motivation from Artificial Intelligence Feedback, September 2023. URL <http://arxiv.org/abs/2310.00166>.
- [19] Martin Klissarov, Mikael Henaff, Roberta Raileanu, Shagun Sodhani, Pascal Vincent, Amy Zhang, Pierre-Luc Bacon, Doina Precup, Marlos C. Machado, and Pierluca D’Oro. MaestroMotif: Skill Design from Artificial Intelligence Feedback, December 2024. URL <http://arxiv.org/abs/2412.08542>.
- [20] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large Language Models are Zero-Shot Reasoners. In *Advances in Neural Information Processing Systems*, May 2022. URL <https://openreview.net/forum?id=e2TBb5y0yFf>.
- [21] Nishanth Kumar, Tom Silver, Willie McClinton, Linfeng Zhao, Stephen Proulx, Tomas Lozano-Perez, Leslie Pack Kaelbling, and Jennifer Barry. Practice Makes Perfect: Planning to Learn Skill Parameter Policies. *Robotics: Science and Systems 2024*, July 2024.
- [22] Do Xuan Long, Duong Ngoc Yen, Anh Tuan Luu, Kenji Kawaguchi, Min-Yen Kan, and Nancy F. Chen. Multi-expert Prompting Improves Reliability, Safety and Usefulness of Large Language Models, November 2024. URL <https://aclanthology.org/2024.emnlp-main.1135/>.
- [23] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback, May 2023. URL <http://arxiv.org/abs/2303.17651>.
- [24] Sumeet Ramesh Motwani, Mikhail Baranchuk, Lewis Hammond, and Christian Schroeder de Witt. A perfect collusion benchmark: How can ai agents be prevented from colluding with information-theoretic undetectability? In *Multi-Agent Security Workshop@ NeurIPS’23*, 2023.
- [25] Sumeet Ramesh Motwani, Mikhail Baranchuk, Martin Strohmeier, Vijay Bolina, Philip Torr, Lewis Hammond, and Christian Schroeder de Witt. Secret collusion among ai agents: Multi-agent deception via steganography. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [26] Alex Gu Wen-Ding Li Fanjia Yan Tianjun Zhang Sida Wang Armando Solar-Lezama Koushik Sen Ion Stoica Naman Jain, King Han. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint*, 2024.
- [27] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. MLGym: A New Framework and Benchmark for Advancing AI Research Agents, February 2025. URL <http://arxiv.org/abs/2502.14499>.
- [28] Alexander Novikov, Ngân Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J R Ruiz, Abbas Mehrabian, M Pawan Kumar, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. May 2025.
- [29] OpenAI. OpenAI o1 System Card, 2024. URL <https://openai.com/index/openai-o1-system-card/>.
- [30] OpenAI. OpenAI o3-mini, 2025. URL <https://openai.com/index/openai-o3-mini/>.
- [31] Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. Recursive Introspection: Teaching Language Model Agents How to Self-Improve. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, November 2024. URL https://openreview.net/forum?id=DRC9pZwBwR&referrer=%5Bthe+profile+of+Yuxiao+Qu%5D%28%2Fprofile%3Fid%3D~Yuxiao_Qu1%29.
- [32] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A Graduate-Level Google-Proof Q&A Benchmark, November 2023. URL <http://arxiv.org/abs/2311.12022>.

- [33] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools, February 2023. URL <http://arxiv.org/abs/2302.04761>.
- [34] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The Prompt Report: A Systematic Survey of Prompting Techniques, June 2024. URL <http://arxiv.org/abs/2406.06608>.
- [35] Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. AgentSquare: Automatic LLM Agent Search in Modular Design Space, November 2024. URL <http://arxiv.org/abs/2410.06153>.
- [36] Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, et al. Language models are multilingual chain-of-thought reasoners. *arXiv preprint arXiv:2210.03057*, 2022.
- [37] Noah Shinn, Federico Cassano, Beck Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, June 2023. URL <http://arxiv.org/abs/2303.11366>.
- [38] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models, October 2023. URL <http://arxiv.org/abs/2305.16291>.
- [39] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*, May 2022. URL https://openreview.net/forum?id=_VjQ1MeSB_J.
- [40] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search, April 2025. URL <http://arxiv.org/abs/2504.08066>.
- [41] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large Language Models as Optimizers, April 2024. URL <http://arxiv.org/abs/2309.03409>.
- [42] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models, May 2023. URL <http://arxiv.org/abs/2305.10601>.
- [43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*, February 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- [44] Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. Gödel Agent: A Self-Referential Framework Helps for Recursively Self-Improvement. October 2024. URL <https://openreview.net/forum?id=dML3XGvWmy>.
- [45] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping Reasoning With Reasoning, May 2022. URL <http://arxiv.org/abs/2203.14465>.
- [46] Eric Zelikman, Qian Huang, Gabriel Poesia, Noah D Goodman, and Nick Haber. Parsel : Algorithmic Reasoning with Language Models by Composing Decompositions. 2023.
- [47] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation, August 2024. URL <http://arxiv.org/abs/2310.02304>.
- [48] Jenny Zhang, Joel Lehman, Kenneth Stanley, and Jeff Clune. OMNI: Open-endedness via Models of human Notions of Interestingness, February 2024. URL <http://arxiv.org/abs/2306.01711>.

附录 A 智能体提示词

主智能体是入口点，负责将请求路由至其他智能体，并将子智能体的结果综合为最终答案。

Main Agent Prompt

```
Your task is to orchestrate sub-agents in order to solve the problem. Here is the problem to solve:
Problem Statement =====
{problem_statement}
End Problem Statement =====
You must now delegate this problem to one, two or however many agents are required to thoroughly solve the
problem.
Ensure you relay the problem statement accurately and completely to the agent. Sub-agents will have access to
the problem statement too, and so it should rarely be necessary to write it out verbatim. In particular, if the
problem statement is very long, you should explicitly NOT write it out again in full. Instead, just give clear
direction, and trust that the sub-agent can also refer to the problem statement.
You have been given access to file and directory viewing tools, since these can help you get your bearings
and direct sub-agents more effectively. These are however meant to help you understand the context in which
you operate. You are intentionally not directly equipped with any tools to conduct substantive work, because
you are just the router, delegator and orchestrator. The tools that your sub-agents' have available are
however listed, and you should carefully refer to these when considering which agent to call next. It is these
sub-agents' job to make the necessary state changes to make progress on the task at hand.
```

A.1 基础子智能体提示词

Main Agent Prompt

```
As a professional and experienced programmer, your approach is to:

1. slow down and don't write files fully end-to-end in one go
2. first understand your context thoroughly:
  2.1 explore the project to locate all the files that could be useful documentation (README.md files, common
  likely MD documentation files, etc)
  2.2 view each of these files, making notes or summaries, and closing irrelevant or long files
  2.3 explore the codebase as it relates to your instructions: find all relevant files, in order to identify
  existing design patterns and conventions
3. (optional) prototype and design before starting coding
  3.1 come up with some simple toy examples in a testing directory
  3.2 use execution feedback to benchmark or compare the approaches
  3.3 synthesise the information and learnings into a final design or solution
4. make code edits
  4.1 identify the most minimal and effective ways to make your required changes
  4.2 observe any existing stylistic conventions
5. test end-to-end
  5.1 Prefer end-to-end testing in testing scripts without test frameworks
  5.2 If this is not an option or the project already uses a testing framework, then use that
  5.3 Ensure your code is valid, hasn't introduced any regressions and works as intended
6. Clean up after yourself
  6.1 Check that all the documentation is still up to date after your changes
  6.2 Clean up any temporary files or changes

NOTE:
- don't create virtual environments
- avoid pytest and mocks if you can; prefer end-to-end scripts
- if the request is clearly exploratory in nature, then you may bypass the rigorous procedure above, and address
  it appropriately
- call your reasoning agent if you are stuck on a tricky algorithmic or mathematical problem, to help you gain
  insight and make progress
```

Main Agent Prompt

Here is the problem you have been asked to solve:

Your problem =====
{self.problem_statement}
=====

Approach:

1. Talk over what the question or problem is asking of you
2. Plan minimum viable solution and validate approach
3. Execute swiftly and adapt based on feedback
4. Verify against original requirements

Key principles:

- Choose simplest viable solution
- Validate early and often
- Adapt swiftly when blocked

Main Agent Prompt

Here is the initial request, provided for context only, that we are ultimately trying to solve:
Initial request:

{self._context.initial_request}

End initial request.

Here is the problem you have been asked to solve:

Problem statement:

{self.problem_to_solve}

End problem statement.

A.2 监督者提示词

Main Agent Prompt

You are an expert system overseer responsible for monitoring agent execution in real-time. You will be shown execution traces as they develop and must make decisions about whether intervention is needed. You are running in a loop.

Your task is to:

1. Analyze the current execution state
2. Determine if you have enough information to make confident assessments
3. Evaluate whether agents are progressing healthily or need intervention
4. Provide thoughtful and constructive steering notifications to running agents if intervention is needed
5. In extreme cases, cancel the execution of an agent and notify its parent of why this is
6. Leave notes for future evaluation iterations if needed

You must provide your analysis in the following structured format:

OVERSEER_EXAMPLES

Here is the current execution trace:

graph_repr

Previous overseer notes (if any)

self._previous_notes if self._previous_notes else 'No notes.'

Key considerations:

- Agents that have already returned cannot receive notifications
- Look at timestamps and event counts to understand execution progress
- Some agents may legitimately take long periods to complete
- Consider both local agent health and global execution progress
- If the agent appears stuck without terminating, remind it that it can exit by generating <COMPLETE><COMPLETE_STOP_TOKEN>
- If, when the agent has been tasked to work on its own code at meta-improvement time, it is attempting to test tools or other features directly (using its own running code), intervene immediately and tell it to either test a new agent end-to-end (python -m workdir.agent -p "<some prompt>") or to create test scripts to test the tools directly.
- You may cancel an agent if it is no longer making progress, wasting resources, or very clearly going down the wrong path or deviating from its instructions. You must make at least one attempt to notify the agent of its wrongdoings before issuing a cancellation command.
- When canceling an agent, you must also notify its parent calling agent of the reasons for the cancellation.
- If the task becomes untenable, after multiple failed attempts, and the agent is appearing to repeat itself, instruct the main agent to use its early_exit tool to exit early.
- Use cancellations sparingly, as they are disruptive and ultimately more costly than guiding the agent to successful completion, since any new sub-agent will often have to start the task from scratch.

The last time you checked was at: self._last_check_time.strftime("%H:%M:%S") if self._last_check_time else "N/A"

The current timestamp is datetime.now().strftime("%H:%M:%S")

Your analysis:

监督者的判断通常采用以下形式：

Main Agent Prompt

```
<OVERSEER_JUDGEMENT>
<making_progress>false</making_progress>
<is_looping>true</is_looping>
<currently_running_agent>agent_d4e5f6g7</currently_running_agent>
<needs_notification_reasoning>The agent has been notified previously about its looping behavior but has not
corrected course. It's time to forcefully cancel this execution.</needs_notification_reasoning>
<needs_notification>true</needs_notification>
<agent_to_notify>agent_b8a796h5</agent_to_notify>
<notification_content>Your software developer agent was looping while trying to install the program, and as a
result I have cancelled it. You might want to start another software developer with clearer instructions about
how to install the program.</notification_content>
<force_cancel_agent>true</force_cancel_agent>
<force_cancel_agent_id>agent_d4e5f6g7</force_cancel_agent_id>
<next_check_type>time</next_check_type>
<next_check_delay>10</next_check_delay>
<notes_for_next_iteration>Forced cancellation of agent_d4e5f6g7 due to unresponsive looping. Monitor parent
agent for proper handling of cancelled child.</notes_for_next_iteration>
<OVERSEER_JUDGEMENT>
```

B 示例迹

Main Agent Prompt

```
EXECUTION TREE
=====
1 main [agent_b8845adf] (134.4s | 95752 tokens (cached 0.57%) | $0.185 | Success)
  [Stats] Events: 1 tool calls, 6 messages
  [Assistant] t+0.1s | "I'll orchestrate the solution to this problem foll..."
  1.3 reasoning_agent [agent_20fccbc1] (12.0s | 5673 tokens (cached 0.00%) | $0.013 | Success)
    [Stats] Events: 1 messages
    [Assistant] t+9.8s | "Below is an explanation that addresses the three p..."
    [Assistant] t+21.1s | "The reasoning agent has provided excellent analy..."
  1.2 solve_problem [agent_11804cif] (17.6s | 54822 tokens (cached 0.68%) | $0.085 | Success)
    [Stats] Events: 3 tool calls, 4 messages
    [Assistant] t+26.1s | "Let me solve this step by step.1. UNDERSTANDING..."
    [Tool] calculate | 0.0s → Success
    [Assistant] t+32.4s | "Therefore there are floor(404.8) = 404 complete..."
    [Tool] return_result | 0.0s → Success
    [Assistant] t+42.1s | "<COMPLETE></COMPLETE>..."
  [Assistant] t+43.7s | "The problem solver has provided a solution..."
  1.1 software_developer [agent_0117a7ef] (69.7s | 208122 tokens (cached 0.68%) | $0.321 | Success)
    [Stats] Events: 8 tool calls, 10 messages
    [Assistant] t+49.3s | "I'll help solve this mathematical game problem thr..."
    [Tool] execute_command | 0.0s → Success
    [Assistant] t+54.6s | "<TOOL_CALL><TOOL_NAME>start_lsp_server</TOOL_NAME>..."
    [Tool] start_lsp_server | 2.2s → Success
    [Assistant] t+59.9s | "Now let's create our main implementation file:..."
    [Tool] overwrite_file | 0.0s → Success
    [Assistant] t+70.4s | "Now let's create a test script to verify our imp..."
    [Tool] overwrite_file | 0.0s → Success
    [Assistant] t+76.8s | "Now let's run our test script to verify everythi..."
    [Tool] execute_command | 0.0s → Success
    [Assistant] t+80.9s | "I notice that there's an issue with our pattern..."
  1.1.1 reasoning_agent [agent_8a69d943] (11.3s | 10231 tokens (cached 0.00%) | $0.017 | Success)
    [Stats] Events: 1 messages
    [Assistant] t+87.2s | "The problem comes from our backward induction func..."
    [Assistant] t+98.5s | "Based on the reasoning agent's analysis, let's u..."
    [Tool] overwrite_file | 0.0s → Success
    [Assistant] t+107.8s | "Now let's run our test script again to verify th..."
    [Tool] execute_command | 0.0s → Success
    [Assistant] t+112.0s | "Perfect! Our implementation now correctly:1. I..."
    [Tool] return_result | 0.0s → Success
    [Assistant] t+116.8s | "<COMPLETE></COMPLETE>..."
  [Assistant] t+119.0s | "Let's verify the software agent's answer since it di..."
  1.4 reasoning_agent [agent_6cd34374] (5.0s | 12025 tokens (cached 0.00%) | $0.016 | Success)
    [Stats] Events: 1 messages
    [Assistant] t+124.1s | "Answer synthesis: Our analysis starts..."
    [Assistant] t+129.1s | "Let's submit our final answer:<TOOL_CALL><TO...>..."
    [Tool] submit_answer | 0.0s → Success
    [Assistant] t+132.1s | "<COMPLETE></COMPLETE>..."

Total Duration: 249.9s
Total Tokens: 407766 (of which cached 232952)
Total Cost: $0.712
```

上述展示了一个智能体解决问题的代表性示例。当前问题是一个数学问题，执行过程始于一个推理智能体（o3 mini），该智能体设计了一种求解方法。随后，该方法被传递给一个“问题求解”智能体，由其尝试求解问题。接着，主智能体调用一个“软件开发”智能体来编写脚本，以对解进行数值验证。软件开发智能体在尝试运行测试脚本后陷入困境，并调用推理智能体来帮助调试问题。应用修复后，测试脚本正确运行，并产生了一个与第一个问题求解器答案不同的解。为整合这两个解，主智能体调用了另一个推理智能体，该智能体支持经验推导出的解，最终由主智能体提交该解作为答案。

Main Agent Prompt

```
EXECUTION TREE
=====
1 main [agent_5e260c4e] (617.5s | 106497 tokens (cached 0.30%) | $0.299 | Success)
  [Stats] Events: 1 tool calls, 5 messages
  [Assistant] t+0.1s | "I'll help orchestrate the solution to this problem..."
  1.1 archive_explorer [agent_534a61ec] (88.6s | 372576 tokens (cached 0.16%) | $1.205 | Success)
    [Stats] Events: 7 tool calls, 10 messages
    [Assistant] t+5.6s | "I'll follow the procedure step by step to analyze..."
    [Tool] compare_agent_iterations | 0.0s → Success
    [Assistant] t+9.7s | "Let's also look at some of the best and worst pe..."
    [Tool] best_problems | 0.0s → Success
    [Assistant] t+13.2s | "Let me also look at some of the worst performing..."
    [Tool] worst_problems | 0.0s → Success
    [Assistant] t+16.7s | "Let me inspect the project documentation..."
    [Tool] open_file | 0.0s → Success
    [Assistant] t+22.1s | "<TOOL_CALL><TOOL_NAME>open_file</TOOL_NAME><TO..."
    [Tool] open_file | 0.0s → Success
    [Assistant] t+27.4s | "Let me also look at the current iteration's desc..."
    [Tool] open_file | 0.0s → Success
    [Assistant] t+33.6s | "Having reviewed the documentation and cu..."
    1.1.2 reasoning_agent [agent_d4553475] (14.8s | 81577 tokens (cached 0.00%) | $0.097 | Success)
      [Stats] Events: 1 messages
      [Assistant] t+44.0s | "I updated description.txt and agent_change_log.md..."
      [Assistant] t+58.1s | "Let me examine this suggestion from the reasoni..."
      1.1.1 reasoning_agent [agent_ccad2709] (5.4s | 81442 tokens (cached 0.02%) | $0.090 | Success)
        [Stats] Events: 1 messages
        [Assistant] t+72.0s | "I've updated both description.txt and agent_change..."
        [Assistant] t+77.4s | "Based on the analysis from both reasoning agents..."
        [Tool] return_result | 0.0s → Success
        [Assistant] t+88.7s | "<COMPLETE></COMPLETE>..."
      [Assistant] t+94.1s | "The archive explorer has made a specific suggest..."
    1.2 software_developer [agent_6345b78d] (331.2s | 1388672 tokens (cached 0.22%) | $4.341 | Success)
      [Stats] Events: 21 tool calls, 22 messages
      [Assistant] t+109.7s | "Let's first look at key documentation files:<T..."
      [Tool] open_file | 0.0s → Success
      [Assistant] t+114.4s | "Now let's look at any existing diff-related code..."
      [Tool] open_file | 0.0s → Success
      [Assistant] t+118.3s | "Let's also look at existing utils and __init__ f..."
      [Tool] open_file | 0.0s → Success
      [Assistant] t+123.1s | "Now that I understand the context better, let me..."
      [Tool] execute_command | 0.0s → Success
      [Assistant] t+274.6s | "Now let's run the tests to verify the implementa..."
      [Tool] execute_command | 0.5s → Success
      [Assistant] t+281.9s | "Oops, looks like I need to fix the test case. Th..."
      ... truncated ...
      [Tool] diff_patch_edit | 0.0s → Success
      [Assistant] t+356.8s | "Now let me run the tests again:<TOOL_CALL><T..."
      [Tool] execute_command | 0.5s → Success
      [Assistant] t+365.3s | "Great! Now that the implementation and tests ar..."
      [Tool] diff_patch_edit | 0.0s → Success
      [Assistant] t+375.6s | "Now let me update description.txt:<TOOL_CALL>..."
      [Tool] open_file | 0.0s → Success
      [Assistant] t+382.6s | "<TOOL_CALL><TOOL_NAME>diff_patch_edit</TOOL_NAM..."
      [Tool] diff_patch_edit | 0.0s → Success
      [Assistant] t+392.4s | "Finally, let me update agent_change_log.md:<..."
      [Tool] diff_patch_edit | 3.6s → Success
      [Assistant] t+415.4s | "Let me verify that I've completed all the requ..."
      [Tool] return_result | 0.0s → Success
      [Assistant] t+428.0s | "<COMPLETE></COMPLETE>..."
    [Assistant] t+434.6s | "The software developer has successfully impleme..."
  1.3 software_developer [agent_ae600007] (167.0s | 194915 tokens (cached 0.54%) | $0.390 | Success)
    [Stats] Events: 9 tool calls, 10 messages
    [Assistant] t+442.9s | "I'll help test the DiffSummarizer implementation b..."
    [Tool] open_file | 0.0s → Success
    [Assistant] t+447.8s | "Now let's create a directory for our test and se..."
    [Tool] execute_command | 0.0s → Success
    [Assistant] t+456.7s | "Now let's execute the agent command to run our t..."
    [Tool] execute_command | 0.0s → Success
    [Assistant] t+460.5s | "I apologize for the error. We need to properly s..."
    [Tool] execute_command | 120.3s → Failed
    [Assistant] t+585.8s | "I see that the command is taking longer than exp..."
    [Tool] open_file | 0.0s → Success
    [Assistant] t+588.9s | "Let's try a direct test of the DiffSummarizer. W..."
    [Tool] execute_command | 0.0s → Success
    [Assistant] t+595.5s | "Now let's run our test script:<TOOL_CALL><TOO..."
    [Tool] execute_command | 0.5s → Success
    [Assistant] t+598.5s | "Perfect! Let's analyze what we've found:1. The..."
    [Tool] return_result | 0.0s → Success
    [Assistant] t+608.3s | "<COMPLETE></COMPLETE>..."
  [Assistant] t+609.8s | "Great! The implementation and testing of the Di..."
  [Tool] submit_answer | 0.0s → Success
  [Assistant] t+615.2s | "<COMPLETE></COMPLETE>..."
```

上述展示了一次典型的元改进运行：智能体首先调用“存档探索者”子智能体，该子智能体可访问多种工具以促进存档分析。注意，这使用了与系统中其他基础智能体完全相同的简单、通用且可扩展的子智能体类。存档探索者比较不同的迭代，查看给定迭代中表现最佳的基准问题以理解成功案例，以及给定迭代中表现最差的基准问题以理解失败模式。随后，它查看一些文件以了解智能体的实现和文档，然后调用两个推理智能体来帮助其对这些信息进行推理。该智能体调用的结果是向主智能体提出关于下一步工作内容的建议。

主智能体随后调用“软件开发”子智能体，并指示其实现该功能。编码子智能体首先查看相关文件和文档，实现该功能，然后进入调试循环，修复其实现中的错误。大约 3 分钟后，它成功使其（自己编写的）所有测试通过，在智能体变更日志中记录该更改，并返回主智能体。

主智能体随后调用第二个软件开发子智能体，以独立验证该功能的实现。该实现经过一个初始问题后通过验证，智能体将新功能工具的描述返回给主智能体，主智能体随后退出。

C 函数调用接口

大语言模型（LLM）函数调用接口允许大语言模型在运行时调用工具和子智能体，而无需预定义控制流。实现这一功能的方法有很多，从约束生成，到在不中断的情况下解析智能体令牌生成流，再到直接依赖大语言模型提供商或推理框架的原生函数或工具调用实现。

为了获得最大的灵活性，我们采用了一种无约束的结构化生成方法，其中大语言模型输出 XML 格式的生成内容，并将闭合标签注册为停止令牌。通过检查停止原因，我们可以检测是否发生了工具调用或智能体调用，解析内容，并运行工具实现。

例如，以下是一个工具调用语法示例：

```
<TOOL_CALL>
<TOOL_NAME>tool_name</TOOL_NAME>
ME> <TOOL_ARGS> <arg1>值
1</arg1> <arg2>值 2</arg2>
</TOOL_ARGS> </TOOL_CALL>
```

其中 `</TOOL_CALL>` 被注册为停止令牌。随后首先解析 `tool_name`，并将其用于索引调用智能体可用的工具集，接着将工具参数解析为字典，并实例化工具类。

我们选择 XML 而非 JSON 作为工具调用格式，因为它不需要转义字符串字面量，这在将文件内容原样作为参数包含时会变得繁琐。

D 附加结果详情

以下为智能体所执行的更新序列，与图 3 中的曲线图相对应。

1. 智能编辑器：实现了一种新型智能编辑器工具，可根据编辑特征智能选择并执行最优文件编辑策略。
2. 快速覆写工具：用于解决文件编辑操作的性能问题，以改进全文件覆写方式，从而减少令牌使用量。
3. 差分增强型智能编辑器：增加了基于差分的智能策略选择，并改进了基于模式的编辑。
4. 简化的差分验证器测试：改善了开发者体验，以提高可维护性。

5. 代码上下文摘要器：新增了一种代码上下文摘要器工具，利用 ripgrep 高效提取并总结代码上下文，旨在提高代码库探索效率。6. 智能编辑器输入规范化器：新增了输入规范化器组件，以提高编辑操作的效率和可靠性。7. 增强的文件编辑验证：实现了一套全面的文件状态跟踪系统，以提高编辑操作效率。8. 最小化差分输出优化：通过最小化差分上下文输出，实现了减少文件编辑操作中令牌使用量的优化。9. 抽象语法树符号定位器：集成了基于抽象语法树的符号定位工具，用于精确的代码导航。10. 优化的文件编辑路由：在主智能体中实现了显式任务类型检测与路由优化，以改进文件编辑任务的处理。11. 增强的文件大小优化：增加了 100KB 的文件大小阈值，以优化大文件处理并减少大改动时的处理开销。12. 混合符号定位器：实现了一种混合符号定位方法，结合了基于 ripgrep 的候选识别与抽象语法树验证。13. 上下文敏感的差分最小化：完成了将上下文敏感的差分最小化集成到智能编辑器工具中。对 Python 代码变更使用抽象语法树解析，对其他文件类型使用基于模式的分析。旨在通过最小化变更上下文来减少令牌使用量。14. 混合符号定位器中的优化抽象语法树解析：在 hybrid_symbol_locator.py 中实现了有针对性的抽象语法树解析，仅处理相关代码区域而非整个文件。